

# Artificial Intelligence (CS311)

## Lecture 1: AI as Search

# Outline of This Lecture

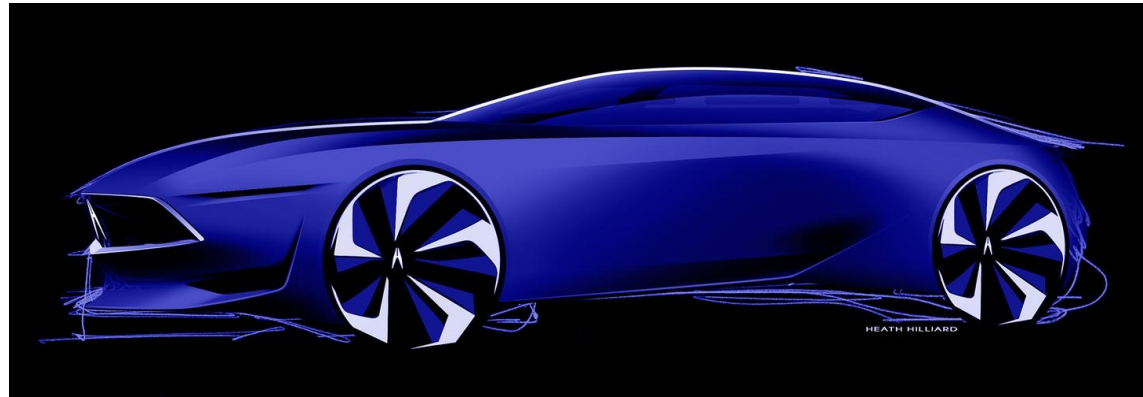
- What is search
- From searching to search tree
- Uninformed Search Methods
- Heuristic (informed) Search
- Further Studies on Heuristics

# What is Search?

- Search is ubiquitous (our thought process can usually be viewed as a search process).
  - To find a satisfactory (user-defined) option from many options



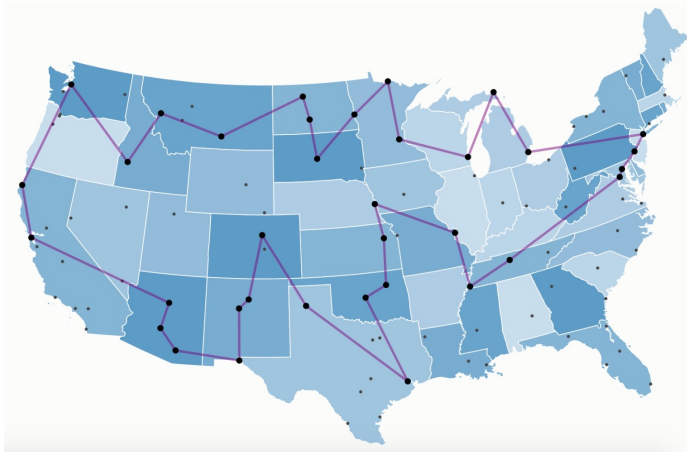
Play Klotski



Design a fancy car

# Technical Question

- **In general**, how could an agent find the satisfactory option?
  - Enumerate and compare, sometimes fine, but in many cases too costly
  - Any smarter approach: don't enumerate
    - to *test* which options? → need a better **organized** search process

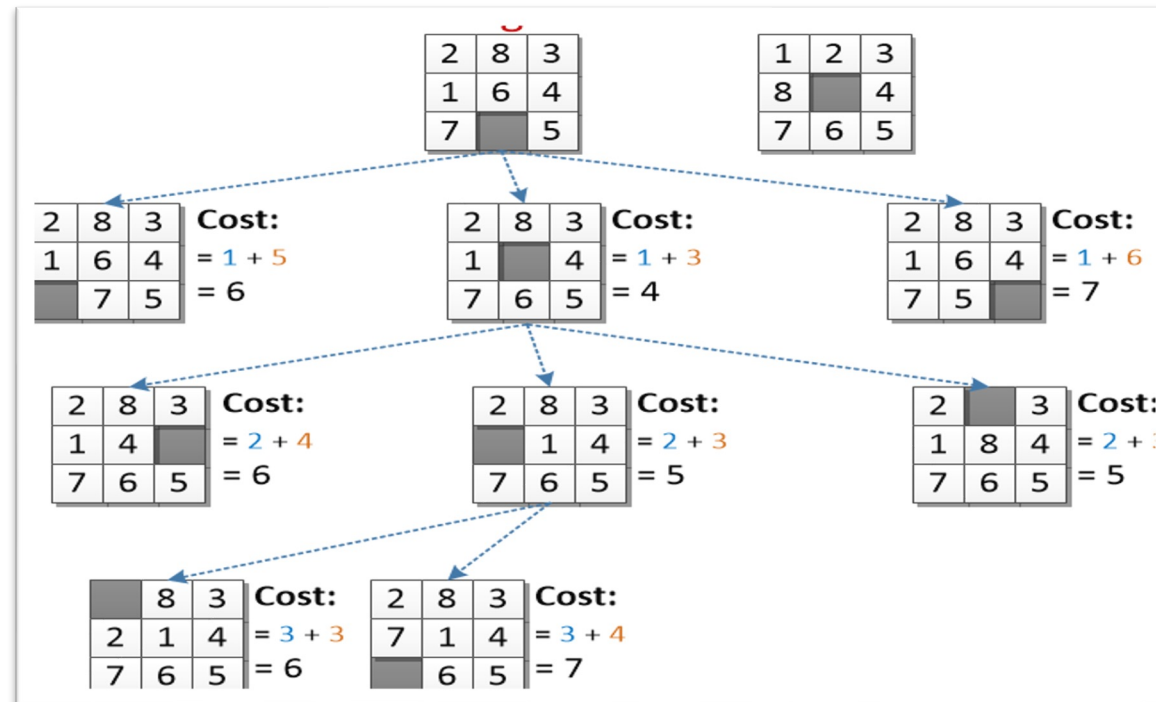


Traveling Salesman Problem: find the shortest path from NY to visit all states of USA and back to NY

- No. of options = 50!

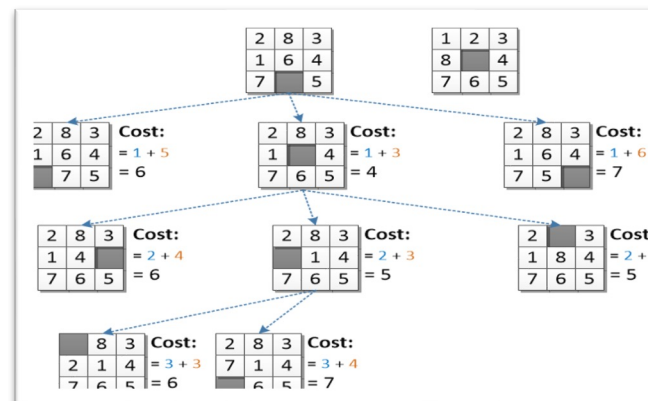
# Search in A Tree

- Search Tree: A universal **representation** to organize the search process



# Search in A Tree

- Search methods are defined by picking the **order of node expansion**.
  - aim to identify the '**best**' solution.
  - differ in how they *explore the space*, i.e., **how they choose the node to expand NEXT**.

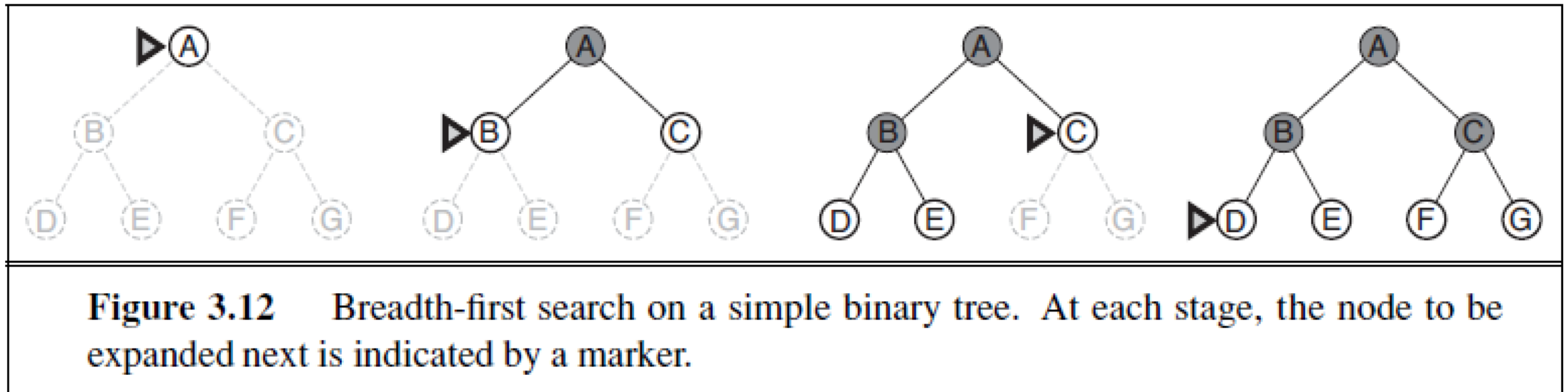


# Un-informed Search Methods

- Use only the information available in the problem definition.
  - Use **NO** problem-specific knowledge.
- 
- Breadth-first search (BFS)
  - Uniform-cost search (UCS)
  - Depth-first search (DFS)
  - Depth-limited search (DLS)
  - Iterative deepening search (IDS)
  - Bidirectional search

# Breadth-First Search (BFS)

- Expand **shallowest** unexpected node.
- Implementation: a FIFO **queue**.

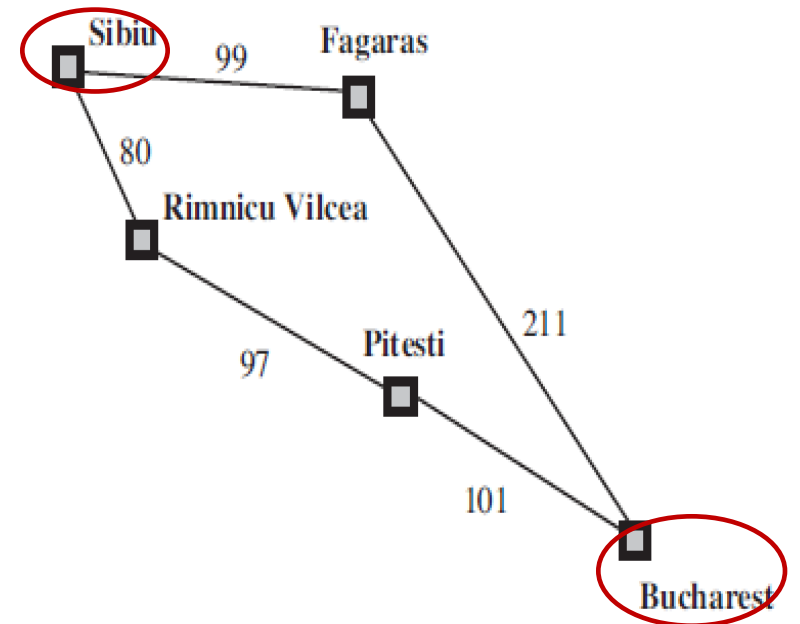


# Uniform-Cost Search (UCS)

- The costs in the search tree may be different.
- Expand the **cheapest** unexpanded node.
- Implementation: a queue ordered by path cost, lowest first.

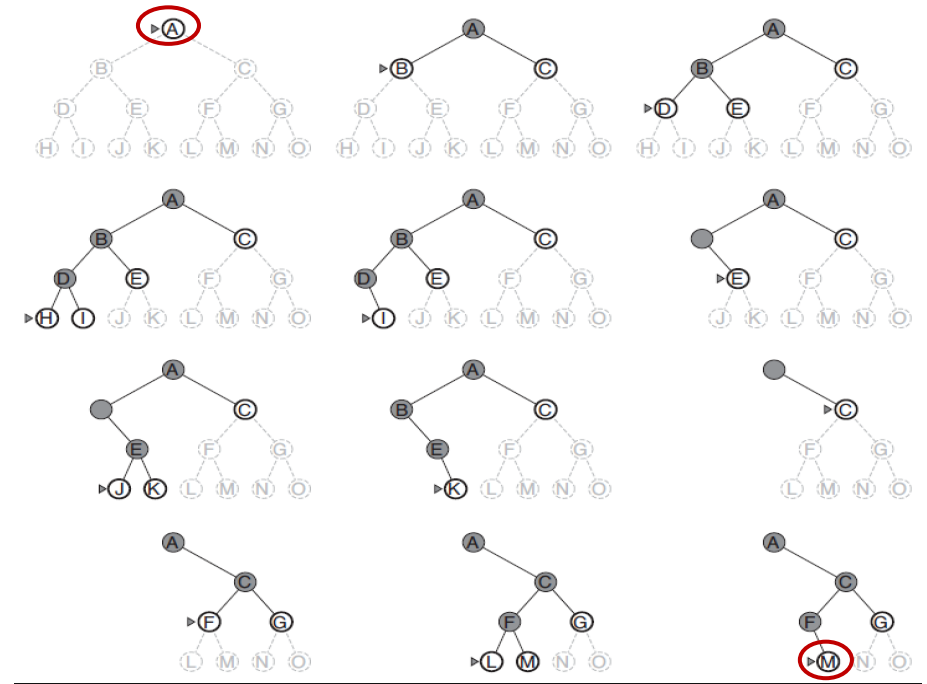
- Task: from **Sibiu** to **Bucharest**.

- [0] {[Sibiu, 0]}
- [1] {[Sibiu→Rimnicu, 80]; [Sibiu→Fagaras, 99]}
- [2] {[Sibiu→Rimnicu→Pitesti, 177]; [Sibiu→Fagaras, 99]}
- [3] {[Sibiu→Rimnicu→Pitesti, 177];  
[Sibiu→Fagaras→Bucharest, 310]}
- [4] {[Sibiu→Rimnicu→Pitesti→Bucharest, 278];  
[Sibiu→Fagaras→Bucharest, 310]}



# Depth-first Search (DFS)

- Expand **deepest** unexpanded node.
- Implementation: LIFO **stack**.
- **Task**: Search from **A** to **M**.
- **Note**: Once a node is expanded, it is removed from memory as soon as all its children are explored.



# Depth-Limited Search (DLS)

- DFS with depth limit  $l$ : nodes at depth  $l$  have no successors.
  - Limit  $l$  is defined based on domain knowledge.
    - e.g. a traveller problem with 20 cities  $\rightarrow l < 20$ .
  - DLS is the variant of DFS.
- DLS overcomes the failure of DFS in infinite-depth space.

# Iterative Deepening Search (IDS)

- Apply **DLS with increasing limits**.
- Combine the benefits of BFS and DFS.

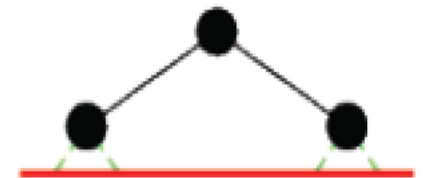
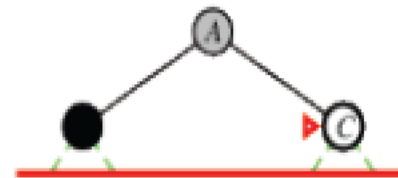
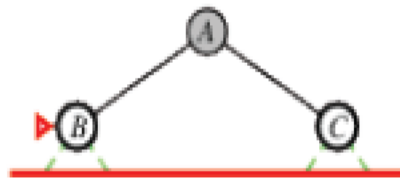
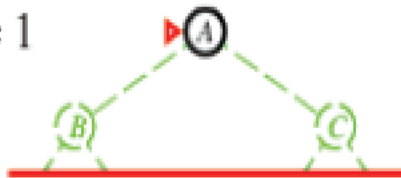
Limit = 0



# Iterative Deepening Search (IDS)

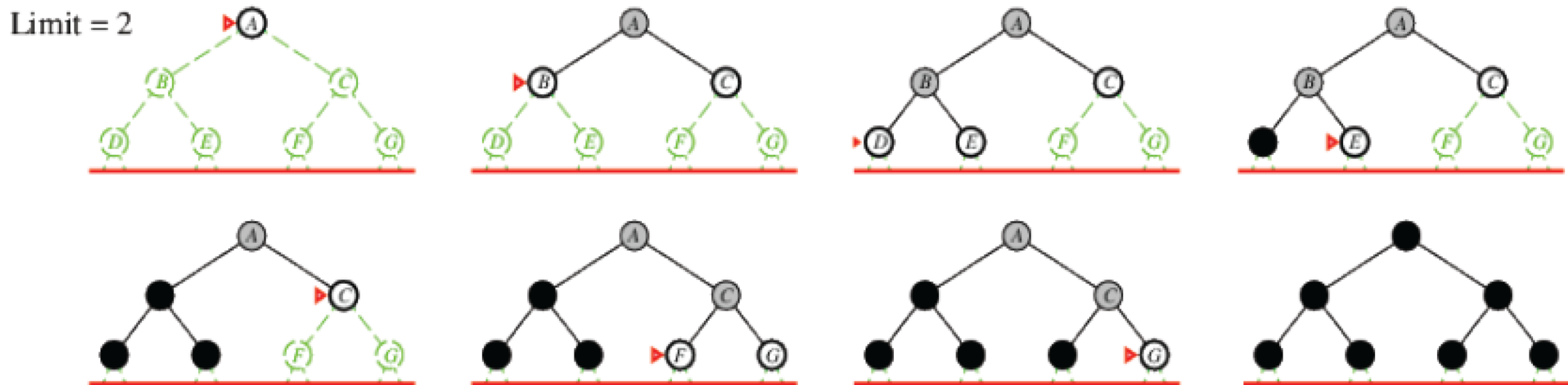
- Apply **DLS with increasing limits**.
- Combine the benefits of BFS and DFS.

Limit = 1



# Iterative Deepening Search (IDS)

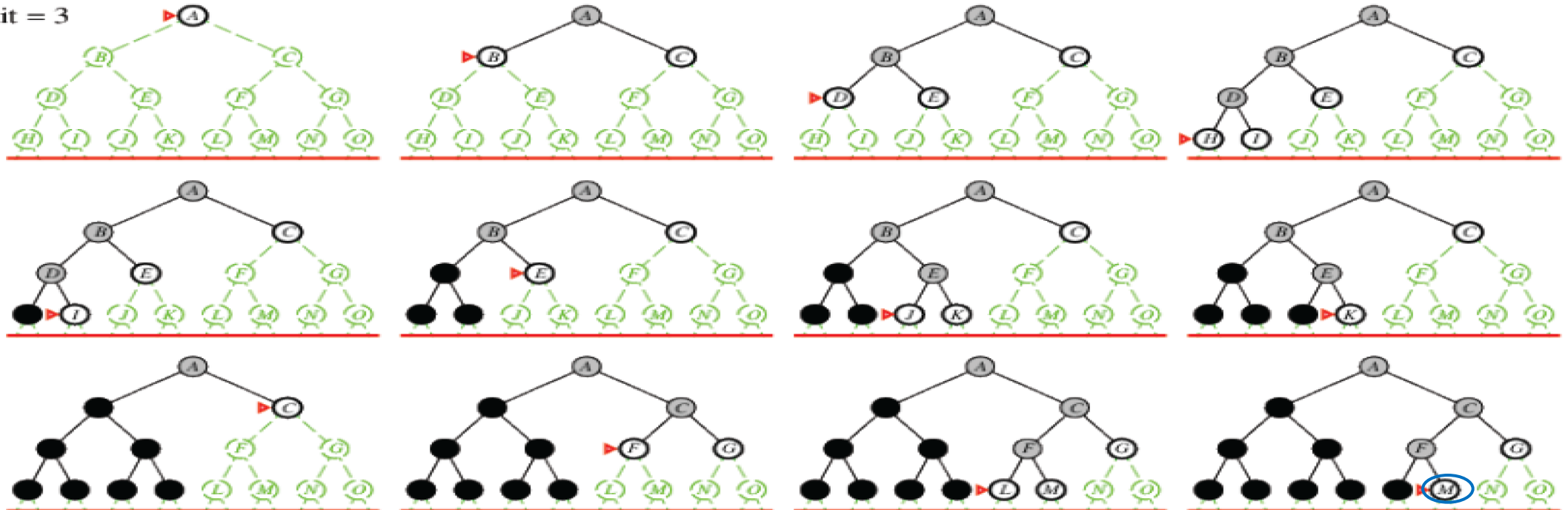
- Apply **DLS with increasing limits**.
- Combine the benefits of BFS and DFS.



# Iterative Deepening Search (IDS)

- Apply **DLS** with increasing limits.
- Combine the benefits of BFS and DFS.

Limit = 3



# Bidirectional Search

- Search from both directions simultaneously.
  - Replace single search tree with two smaller sub trees.
  - Forward tree: forward search from source to goal.
  - Backward tree: backward search from goal to source.
- Disadvantage: Not always applicable
  - Reversible actions?
  - Explicitly stated goal state?

# What is A Good Search Method?

- **Completeness:** Does it always find a solution (a satisfactory option) if it exists?
- **Optimality:** Does it always find the least-cost solution?
- **Time complexity:** # nodes generated/expanded.
- **Space complexity:** maximum #nodes in memory.

# What is A Good Search Method?

In general, time and space complexity depend on:

- $b$  – maximum # successors of any node in search tree.
- $d$  – depth of the least-cost solution.
- $m$  – maximum length of any path in the state space.

# Un-informed Search Methods

Generally, not efficient, not that intelligent

$b$  – maximum # successors of any node in search tree.  
 $d$  – depth of the least-cost solution.  
 $m$  – maximum length of any path in the state space.

| PF Metric        | Breadth-first Search                         | Uniform-cost Search                   | Depth-first Search            | Depth-limited Search | Iterative Deepening                          | Di-directional Search                           |
|------------------|----------------------------------------------|---------------------------------------|-------------------------------|----------------------|----------------------------------------------|-------------------------------------------------|
| <b>Complete?</b> | Yes*, if $b$ is finite.                      | Yes*, if step costs $\geq \epsilon$ . | No, infinite loops can occur. | No. (Eps. $l < d$ )  | Yes                                          | Yes*, if BFS used for both search.              |
| <b>Optimal?</b>  | Yes*, if costs on the edge are non-negative. | Yes                                   | No,                           | No                   | Yes*, if costs on the edge are non-negative. | Yes*, if BFS is used & paths have uniform cost. |
| <b>Time?</b>     | $O(b^{d+1})$                                 | $O(b^{1+\lceil C^*/\epsilon \rceil})$ | $O(b^m)$                      | $O(b^l)$             | $O(b^d)$                                     | $O(b^{d/2})$                                    |
| <b>Space?</b>    | $O(b^{d+1})$                                 | $O(b^{1+\lceil C^*/\epsilon \rceil})$ | $O(bm)$                       | $O(bl)$              | $O(bd)$                                      | $O(b^{d/2})$                                    |

# From Basic Search to Heuristic Search

- Uninformed search uses **NO** domain (problem-specific) knowledge.
- Which node to expand is decided arbitrarily (sometimes randomly).
- Heuristic search **uses** domain knowledge.
  - **Heuristics**: which part of the search space to explore?  $\Rightarrow$  help direct the search.

# Heuristic Search

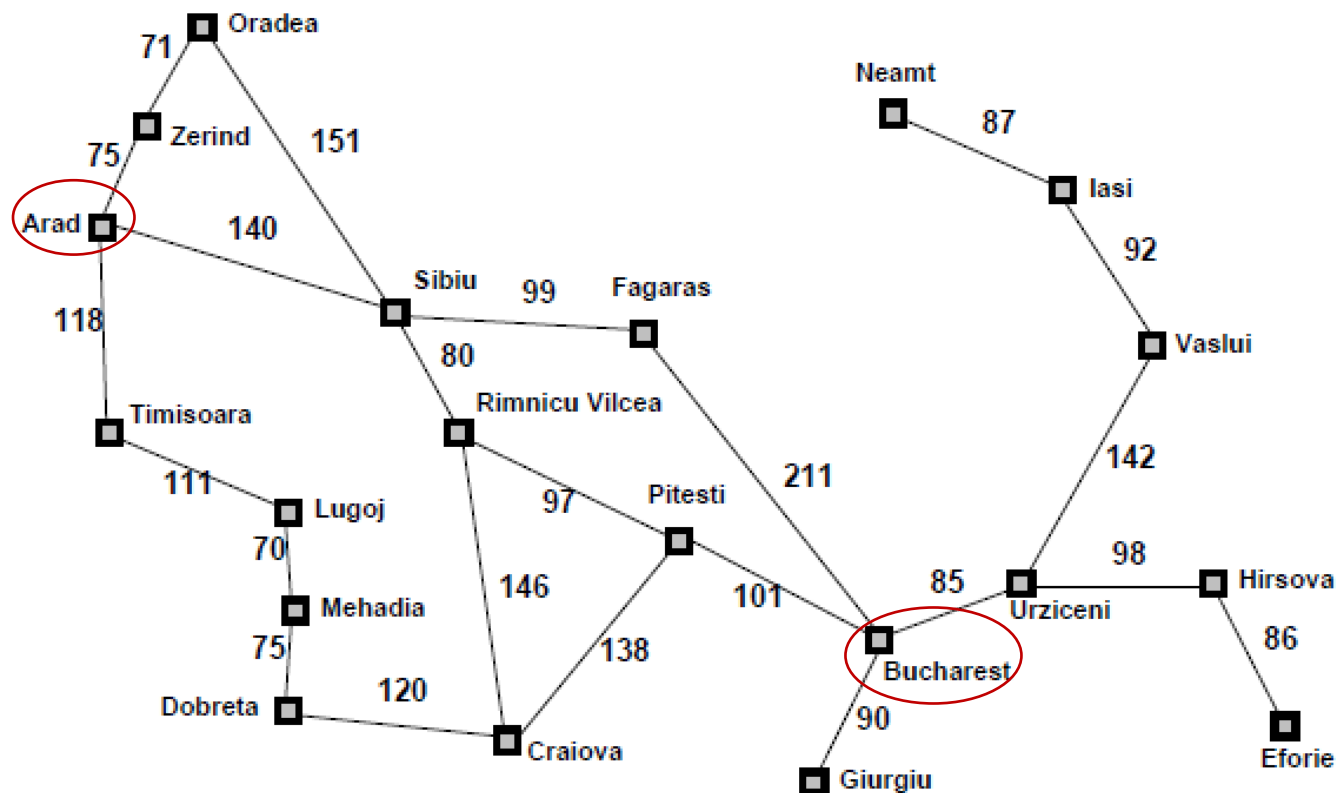
- How to make use of domain knowledge?
  - Define an evaluation function  $f(x)$  at node  $x$ .
  - node  $x$  with the **lowest**  $f(x)$  is expanded first
- Define a good  $f$  endows with some **intelligence** → A little AI now.

# How to Design the Evaluation Function?

- Critical! But depends on your own...
- Basically, introduce a *heuristic function*  $f(x)$  to **estimate** the cheapest cost from  $x$  to the goal state.
  - (1)  $f(x) = 0$  if  $x$  is the goal node.
  - (2) nonnegative.
  - (3) **problem-specific**.
- Interpretation: Expand the node that **seems the best**
  - a generalization of UCS.

# A not-so-good illustration

## Romania with step costs in km



| Straight-line distance to Bucharest |     |
|-------------------------------------|-----|
| Arad                                | 366 |
| Bucharest                           | 0   |
| Craiova                             | 160 |
| Dobreta                             | 242 |
| Eforie                              | 161 |
| Fagaras                             | 178 |
| Giurgiu                             | 77  |
| Hirsova                             | 151 |
| Iasi                                | 226 |
| Lugoj                               | 244 |
| Mehadia                             | 241 |
| Neamt                               | 234 |
| Oradea                              | 380 |
| Pitesti                             | 98  |
| Rimnicu Vilcea                      | 193 |
| Sibiu                               | 253 |
| Timisoara                           | 329 |
| Urziceni                            | 80  |
| Vaslui                              | 199 |
| Zerind                              | 374 |

$$f(x) = h_{SLD}(x)$$

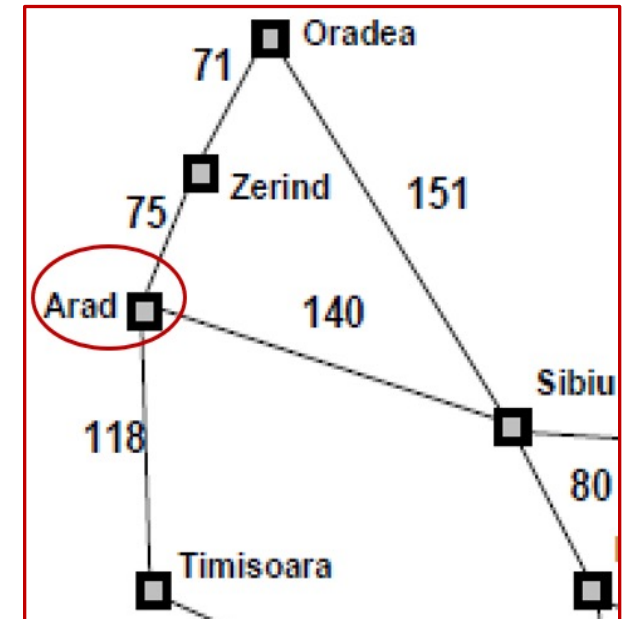
**domain-  
knowledge**

Example:  $h_{SLD}(x)$  = straight-line distance from node  $x$  to goal.

# A not-so-good illustration

Straight-line distance  
to Bucharest

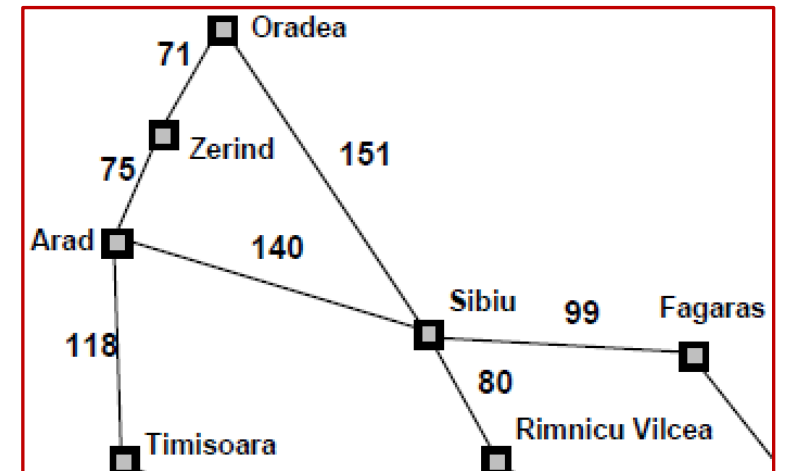
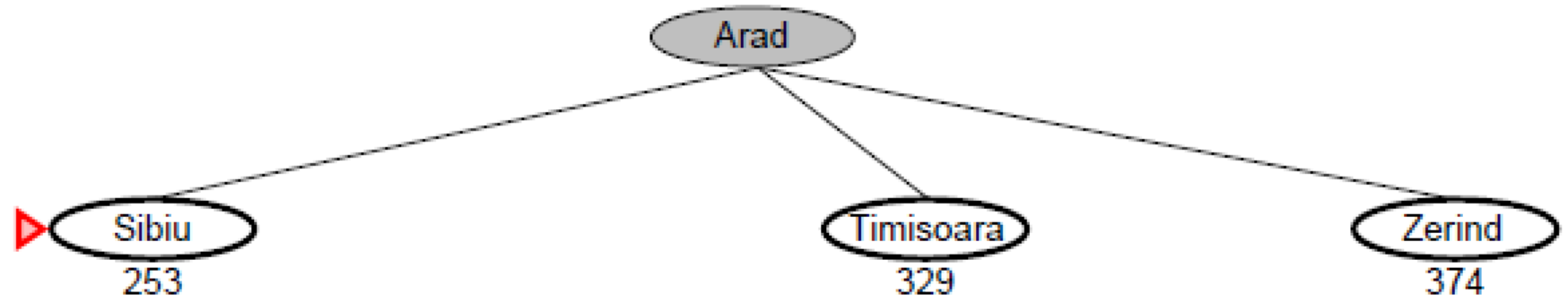
|                |     |
|----------------|-----|
| Arad           | 366 |
| Bucharest      | 0   |
| Craiova        | 160 |
| Dobreta        | 242 |
| Eforie         | 161 |
| Fagaras        | 178 |
| Giurgiu        | 77  |
| Hirsova        | 151 |
| Iasi           | 226 |
| Lugoj          | 244 |
| Mehadia        | 241 |
| Neamt          | 234 |
| Oradea         | 380 |
| Pitesti        | 98  |
| Rimnicu Vilcea | 193 |
| Sibiu          | 253 |
| Timisoara      | 329 |
| Urziceni       | 80  |
| Vaslui         | 199 |
| Zerind         | 374 |



# A not-so-good illustration

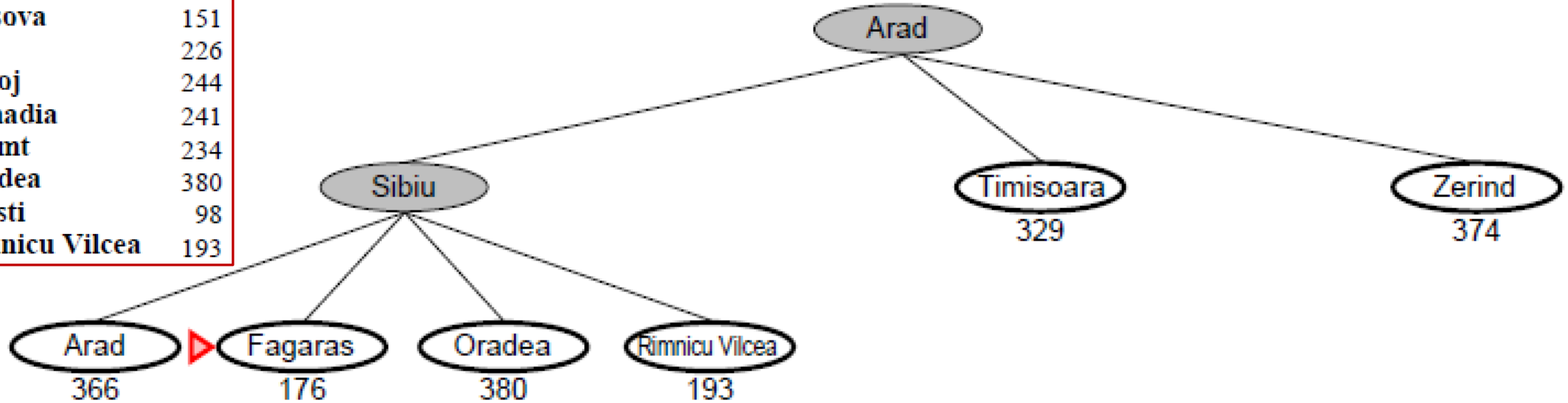
Straight-line distance  
to Bucharest

|                |     |
|----------------|-----|
| Arad           | 366 |
| Bucharest      | 0   |
| Craiova        | 160 |
| Dobreta        | 242 |
| Eforie         | 161 |
| Fagaras        | 178 |
| Giurgiu        | 77  |
| Hirsova        | 151 |
| Iasi           | 226 |
| Lugoj          | 244 |
| Mehadia        | 241 |
| Neamt          | 234 |
| Oradea         | 380 |
| Pitesti        | 98  |
| Rimnicu Vilcea | 193 |
| Sibiu          | 253 |
| Timisoara      | 329 |
| Urziceni       | 80  |
| Vaslui         | 199 |
| Zerind         | 374 |

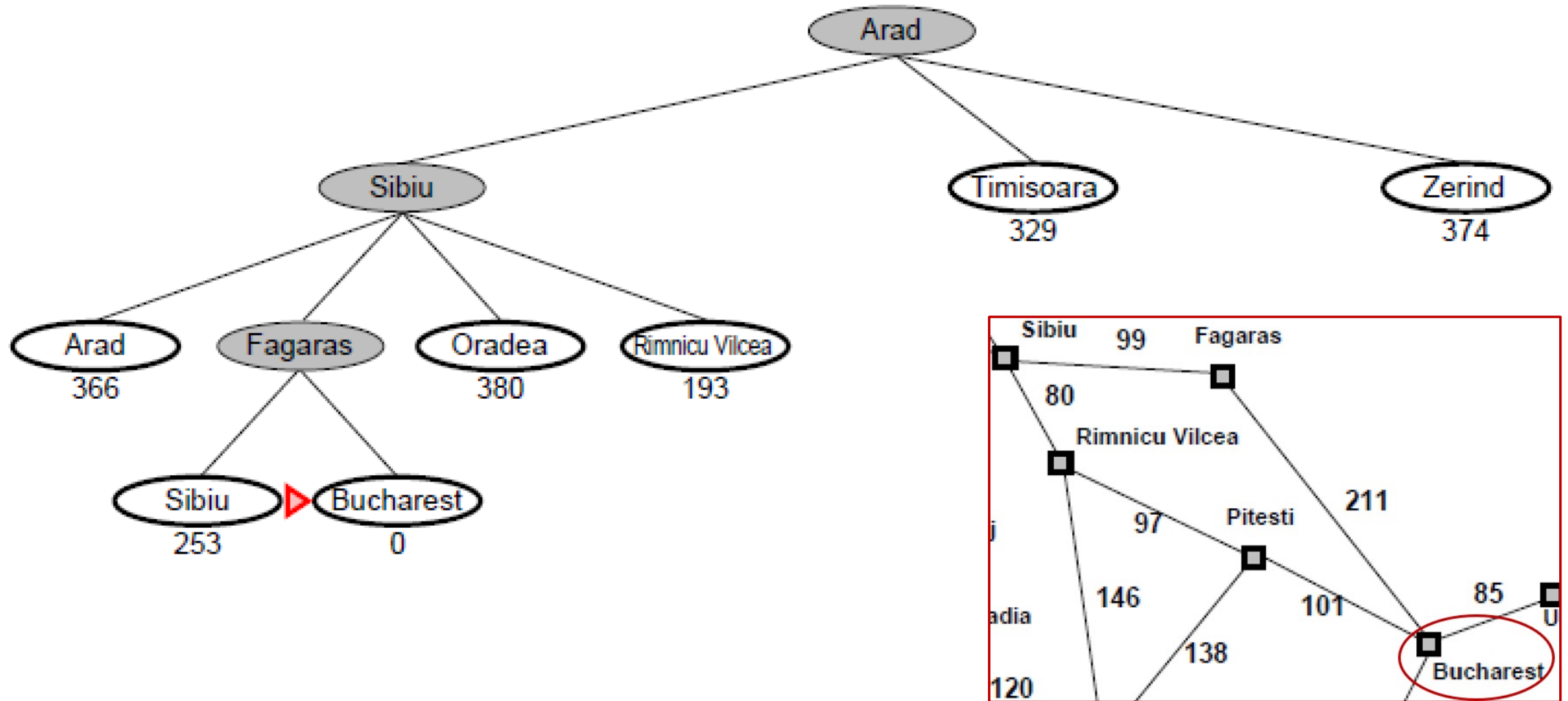


# A not-so-good illustration

|                |     |
|----------------|-----|
| Fagaras        | 178 |
| Giurgiu        | 77  |
| Hirsova        | 151 |
| Iasi           | 226 |
| Lugoj          | 244 |
| Mehadia        | 241 |
| Neamt          | 234 |
| Oradea         | 380 |
| Pitesti        | 98  |
| Rimnicu Vilcea | 193 |



# A not-so-good illustration



# PF Metrics

- $b$  – maximum branching factor of the search tree.
- $d$  – depth of the least-cost solution.
- $m$  – maximum depth of the state space.

- **Complete?** Yes in finite space with repeated-state checking.
- **Optimal?** No. (due to the *bad* heuristic function)
- **Time?**  $O(b^m)$ , but a good heuristic can give drastic improvement.
- **Space?**  $O(b^m)$ , keep all nodes in memory.

# A\* Search

- Expand the node  $n$  that has the minimal  $f(x) = h(x) + g(x)$ 
  - $g(x)$ : cost so far to reach  $x$ .
    - avoid expanding paths that are already expensive.
  - $h(x)$ : estimated cost to goal from  $x$ .
  - $f(x)$ : estimated total cost of path through  $x$  to goal.

# A\*: PF Metrics

- Complete? Yes.
- Optimal? Yes\*, if  $h$  is **admissible**.
- Time?  $O(b^d)$ .
- Space?  $O(b^d)$ , keep every node in memory.

- $b$  – maximum branching factor of the search tree.
- $d$  – depth of the least-cost solution.
- $m$  – maximum depth of the state space.

# Admissible Heuristic

- Heuristic function  $h$  is **admissible** if:

$$\forall x \rightarrow h(x) \leq h^*(x), \quad h^*(x): \text{ the true cost from } x \text{ to goal.}$$

- A\* with  $h$  is optimal if  $h$  is admissible.

**Notation:**  $S$  – start,  $G$  – goal,  $x$  – a node on optimal path,  $x'$  – a non-optimal goal,  $c^*$  – cost of the optimal path.

[Proof] A\* is always finds the optimal path  $\Leftrightarrow$  A\* always pick  $x$  over  $x' \Leftrightarrow f(x) < f(x')$ .

**Known:**  $h$  is admissible  $\Rightarrow h(x) < c^*(x, G)$ . (%  $G$  is a goal state)

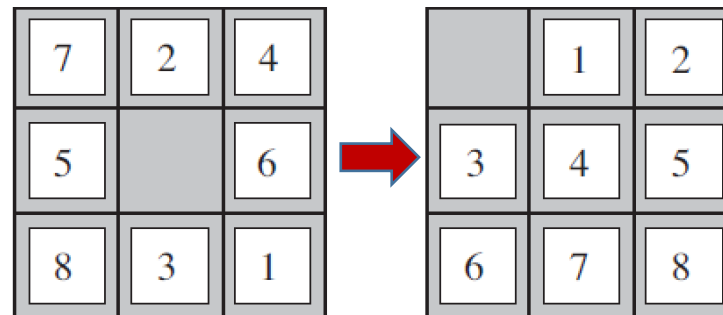
①  $f(x') = g(x') + h(x') = g(x') + 0 > c^*$ . (%  $x'$  is the goal node &  $c^*$  the smallest cost)

②  $f(x) = g(x) + h(x) < g(x) + c^*(x, G) = c^*(S, x) + c^*(x, G) = c^*$ .

**Conclusion:** combine ① & ②  $\Rightarrow f(x) < f(x')$   $\square$

# Admissible Heuristic: Examples

- Route Planning:  $h_{SLD}(n)$ : Admissible, since no solution path will ever shorter than straight-line connection.
- Eight-puzzle:  $h_{mis}(n) = \# \text{misplaced tiles} \in [0,8]$ : Admissible.



# Search Efficiency of Admissible Heuristic

- In general, an  $A^*$  with an admissible heuristic
  - may expands all nodes with  $f(x) \leq c^*$  (already a reduction compared to enumeration)
  - output the optimal solution.
- Different heuristic functions could lead to **huge difference** in efficiency.

| $d$ | Search Cost (nodes generated) |            |            | Effective Branching Factor |            |            |
|-----|-------------------------------|------------|------------|----------------------------|------------|------------|
|     | IDS                           | $A^*(h_1)$ | $A^*(h_2)$ | IDS                        | $A^*(h_1)$ | $A^*(h_2)$ |
| 2   | 10                            | 6          | 6          | 2.45                       | 1.79       | 1.79       |
| 4   | 112                           | 13         | 12         | 2.87                       | 1.48       | 1.45       |
| 6   | 680                           | 20         | 18         | 2.73                       | 1.34       | 1.30       |
| 8   | 6384                          | 39         | 25         | 2.80                       | 1.33       | 1.24       |
| 10  | 47127                         | 93         | 39         | 2.79                       | 1.38       | 1.22       |
| 12  | 3644035                       | 227        | 73         | 2.78                       | 1.42       | 1.24       |
| 14  | –                             | 539        | 113        | –                          | 1.44       | 1.23       |
| 16  | –                             | 1301       | 211        | –                          | 1.45       | 1.25       |
| 18  | –                             | 3056       | 363        | –                          | 1.46       | 1.26       |
| 20  | –                             | 7276       | 676        | –                          | 1.47       | 1.27       |
| 22  | –                             | 18094      | 1219       | –                          | 1.48       | 1.28       |
| 24  | –                             | 39135      | 1641       | –                          | 1.48       | 1.26       |

Data are averaged over 100 instances of the 8-puzzle problem.  
See Fig. 3.29 of the AIAMA book.

# Search Efficiency of Admissible Heuristic

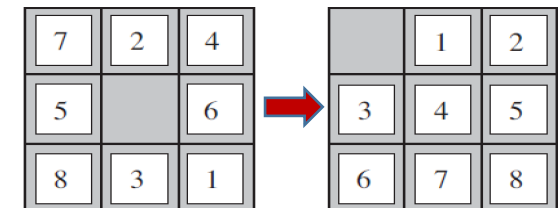
- For admissible  $h_1$  and  $h_2$ , if  $h_2(x) \geq h_1(x), \forall x$ , then  $h_2$  **dominates**  $h_1$  and is **more efficient** for search.
  - The closer of the estimated cost to the true cost, the less nodes will be tested.
- Two extreme examples
  - (1) The **trivial**  $h_0(x) = 0$ : No help for searching  $\Rightarrow$  Not efficient.
  - (2) The **perfect**  $h^*(x)$  = the true cost from  $x$  to the goal: lead directly to the best path

# How to Design Admissible Heuristics

## Consider relaxed problems

For eight puzzle:

- **Rule:** A tile can only move to the adjacent empty square.
- **Relaxed rules:**
  - R1: A tile can move anywhere  $\Rightarrow h_{mis}(n) = \#(\text{misplaced tiles})$ .
  - R2: A tile can move one step in any direction regardless of an occupied neighbour  $\Rightarrow h_{1stp}(n) = \#(1\text{-step move})$  to reach goal.
  - $h_{mis}$  and  $h_{1stp}$  are admissible.
- **Note:** optimal solutions with R1, R2 are **easier** to find.

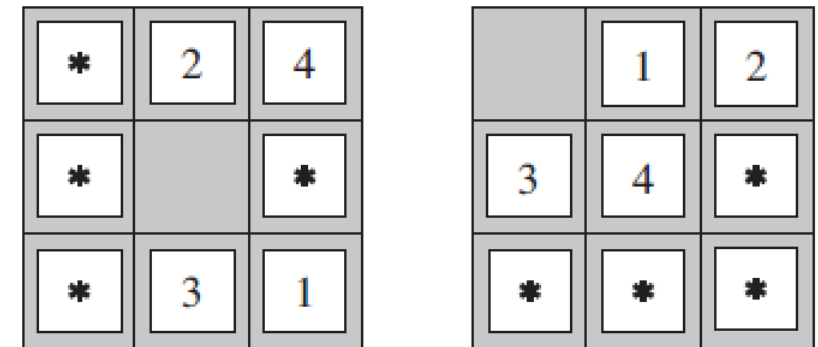


# How to Design Admissible Heuristics

## Consider sub-problems

For eight puzzle:

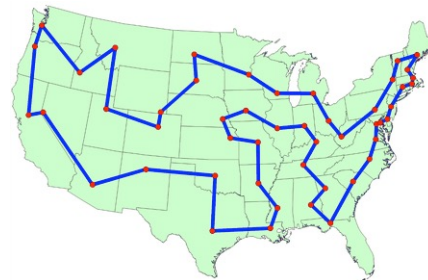
- **Subproblem**: See Fig.1.
  - **Task**: get tiles 1, 2, 3 and 4 into their correct positions.
  - **Relaxation**: move them disregard the other tiles.
- **Theory**:  $\text{cost}^*(\text{subproblem}) < \text{cost}^*(8\text{-puzzle})$ .



A sub-problem of 8-puzzle.

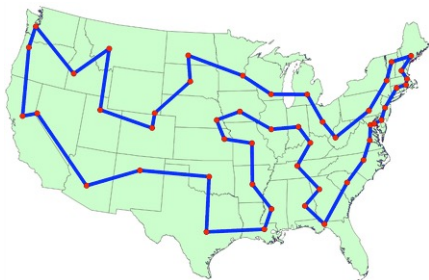
# Summary of Heuristic Search

- We can obtain INTELLIGENCE through enumeration, if cost is not a problem.
- Heuristic search is needed because cost IS an issue
  - Instead of enumerate, try a part of all possibilities/options.
  - We need some way to “generate” solutions to try since it is usually impossible to store all options in advance.
  - Tree structure is a natural choice if a problem could be formulated as a multi-step decision making problem.



# Summary of Heuristic Search

- Leverage on **problem-specific knowledge** to introduce **search biases** into a tree search process, to **more efficiently** obtain a **desired solution** (than un-informed search).
- Search bias: among many options, which to test in the next step?
- Completeness and Optimality is less emphasized in practice than in our lectures, for:
  - Very difficult (if not impossible) to design a good admissible heuristic.
  - For many problems, optimality could **NOT** be **guaranteed** with **affordable time complexity**.



The Christofides' algorithm has an **approximation ratio of  $3/2$**  and **time complexity of  $O(n^2 \log(n))$**  for the metric Traveling Salesman Problem.

To be continued